BotMM « Bot Mohsen et Mamadou » :

Le lien pour ma vidéo : https://youtu.be/1Rd5saVa5yU

Objectif : Développement d'un Bot Automatisé pour Telegram

Résumé :

Ce projet consiste à la création d'un bot automatisé sur la plateforme sociale Telegram, capable de traduire et de transcrire des messages audios instantanément. L'objectif est de simplifier les interactions entre les membres au cas où il y a une diversité de langue sur la plateforme. Les principaux objectifs incluent l'automatisation des processus de traduction et de transcription, l'amélioration de l'expérience utilisateur, et l'application aux bonnes pratiques à long terme (suite de projet dans l'avenir).

Contexte :

Les bots jouent un rôle crucial dans les plateformes sociales et de communication. Ils automatisent des tâches répétitives et enrichissent les fonctionnalités disponibles pour les utilisateurs. Disponibles 24 heures sur 24 et 7 jours sur 7, ils fonctionnent sans interruption, contrairement aux humains, et permettent de réduire les coûts liés à certaines tâches répétitives. C'est exactement ce que propose notre bot de traduction et de transcription : il facilite la communication multilingue en traduisant instantanément les messages, favorisant ainsi les échanges internationaux. De plus, les utilisateurs n'ont plus besoin de recourir à des outils externes pour traduire ou transcrire des messages. Le bot intègre ces fonctionnalités directement dans la plateforme, simplifiant et accélérant les interactions.

Ces fonctionnalités seront intégrées à Telegram, une plateforme de messagerie instantanée populaire et accessible à un grand nombre d'utilisateurs. Grâce à sa popularité garantit que notre solution pourra atteindre un public varié.

Conception et Fonctionnalité :

Choix de la Plateforme :

Les raisons qui nous ont motivés à choisir Telegram pour notre bot sont qu'il permet une interaction fluide et intuitive avec les utilisateurs. Cette dernière est une plateforme de messagerie instantanée largement utilisée dans le monde entier, avec une base d'utilisateurs diversifiés. De plus, Telegram est reconnu pour sa sécurité, sa rapidité et sa flexibilité, offrant des fonctionnalités avancées comme les groupes, les canaux et le support des médias variés.

Notre bot de traduction et de transcription s'adresse à un public varié, répondant à des besoins spécifiques. Il vise notamment les utilisateurs internationaux qui communiquent dans différentes langues, les professionnels et équipes multilingues ayant besoin d'outils pour collaborer efficacement, et les personnes sourdes ou dans des environnements bruyants qui préfèrent lire des transcriptions de messages vocaux. De plus, il sert les apprenants de langues en leur permettant de comparer des messages originaux avec leurs traductions.

Description des Fonctionnalités :

Notre bot sur Telegram intègre des fonctionnalités essentielles pour simplifier la communication. Il propose une transcription des messages vocaux en texte, offrant une alternative pratique pour ceux qui préfèrent lire ou ne peuvent pas écouter. Ainsi, Il propose une traduction instantanée des messages, permettant aux utilisateurs de converser dans différentes langues. Les commandes multilingues rendent l'interaction intuitive. Enfin, la personnalisation des langues permet aux utilisateurs de définir leurs préférences, adaptant le bot à leurs besoins spécifiques.

Architecture :

L'architecture repose sur l'API Telegram pour la communication entre l'utilisateur et le bot. Le bot, développé avec la bibliothèque python-Telegram-bot, traite les messages en temps réel. Pour la traduction nous avons utilisé la bibliothèque translate et la bibliothèque speech_recognition pour la transcription. Les données sont ensuite renvoyées à l'utilisateur via Telegram. Une base de données peut être utilisée pour stocker les préférences des utilisateurs, comme leurs langues préférées, afin de personnaliser les réponses.

Implémentation Technique

1. **Technologies et Bibliothèques :**
   Telegram : La bibliothèque telegram est utilisée pour interagir avec l'API Telegram. Elle permet de gérer les messages entrants et sortants, les utilisateurs, les chats, et bien plus encore. C'est la base de notre bot Telegram.
   Telegram.ext : La sous-bibliothèque telegram.ext fournit des extensions et des outils supplémentaires pour faciliter la gestion des commandes, des messages, et des filtres. Elle simplifie la création de bots en fournissant une structure organisée pour les handlers.

Speech_recognition : La bibliothèque speech_recognition est utilisée pour convertir l'audio en texte. Dans notre bot, elle permet de transcrire les messages vocaux envoyés par les utilisateurs.
Pydub : La bibliothèque pydub est utilisée pour manipuler les fichiers audios. Dans notre cas, elle convertit les fichiers audio OGG en WAV, un format supporté par la bibliothèque speech_recognition.
Translate : La bibliothèque translate est utilisée pour traduire le texte d'une langue à une autre. Dans ton bot, elle permet de traduire le texte transcrit à partir des messages vocaux.
Os : La bibliothèque standard os fournit des fonctionnalités pour interagir avec le système d'exploitation. Elle est utilisée pour des tâches comme la gestion des fichiers et des variables d'environnement.
Logging : La bibliothèque standard logging est utilisée pour générer des logs. Les logs sont essentiels pour déboguer et surveiller l'exécution de notre bot, en enregistrant les événements, les erreurs, et autres informations utiles.
Nest_asyncio : La bibliothèque nest_asyncio est utilisée pour patcher l'asyncio event loop afin de permettre son utilisation dans des environnements comme Jupyter Notebook. Cela évite les erreurs liées aux boucles d'événements lorsqu'on utilise des librairies asynchrones.
Dotenv : La bibliothèque dotenv permet de charger les variables d'environnement depuis un fichier « .env ». Cela améliore la sécurité en évitant de hardcoder les informations sensibles. Dans notre cas, il s'agit de notre API telegram.

2. Développement :

- **Etape 1 : Création du Bot de base**
  Initialiser le bot Telegram et implémenter une commande de démarrage « /start ».

```
In [ ]:  # Configure logging
         logging.basicConfig(
             format='%(asctime)s - %(name)s - %(levelname)s - %(message)s',
             level=logging.INFO
         )
         logger = logging.getLogger(__name__)
         def start(update: Update, context: CallbackContext) -> None:
             update.message.reply_text('Bonjour! Envoyez-moi un message vocal.')
```

- **Etape 2 : Téléchargement, Conversion et Transcription de l'audio. Gestion des erreurs.**

Permettre au bot de recevoir et de convertir des messages vocaux et aussi de les transcrire.

```python
async def transcribe_audio(self, update: Update, context: CallbackContext) -> None:
        """Handle incoming voice messages, transcribe and translate them."""
        try:
            # Download the audio file
            voice_file = await update.message.voice.get_file()
            voice_path = 'voice_message.ogg'
            wav_path = 'voice_message.wav'
            await update.message.reply_text("Traitement de votre message vocal...")
            # Download and convert the file
            await voice_file.download_to_drive(voice_path)
            logger.info('Voice message downloaded successfully.')
            # Convert OGG to WAV
            audio = AudioSegment.from_ogg(voice_path)
            audio.export(wav_path, format="wav")
            logger.info('Audio converted to WAV successfully.')
            # Transcribe the audio
            with sr.AudioFile(wav_path) as source:
                audio_data = self.recognizer.record(source)
                try:
                    # Transcribe
                    original_text = self.recognizer.recognize_google(audio_data)
                    logger.info('Transcription successful.')
                    response = (f"📝 Transcription originale:\n{original_text}\n\n")
                    await update.message.reply_text(response)
                except sr.UnknownValueError:
                    await update.message.reply_text(
                        "Désolé, je n'ai pas pu comprendre l'audio. Assurez-vous que l'enregistrement est clair."
```

- **<u>Etape 3 :</u> Traduction du texte transcrit**

  Traduire le texte transcrit d'une langue à une autre.

```python
def translate_text(self, text: str) -> str:
    """Translate text to French."""
    try:
        translation = self.translator.translate(text)
        return translation
    except Exception as e:
        logger.error(f"Translation error: {e}")
        return "Erreur de traduction"
```

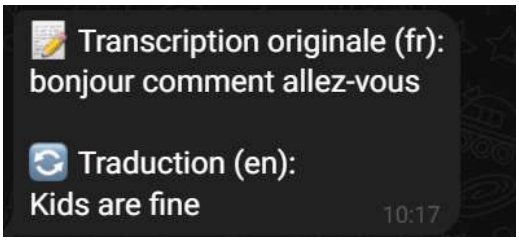
- **<u>Etape 4 :</u> Ajout des boutons de choix de langues**

  Permettre à l'utilisateur de choisir sa langue et celle dans laquelle il souhaite traduire son message.

```python
# Menu de choix de langue
keyboard = [
    [
        InlineKeyboardButton("English", callback_data='en'),
        InlineKeyboardButton("Français", callback_data='fr'),
        InlineKeyboardButton("Español", callback_data='es'),
        InlineKeyboardButton("العربية", callback_data='ar'),
        InlineKeyboardButton("Deutsch", callback_data='de')
    ]
]
reply_markup = InlineKeyboardMarkup(keyboard)
await update.message.reply_text('Choose your language:', reply_markup=reply_markup)
```

3. <u>Tests et Validation :</u>

Pour tester le fonctionnement et l'efficacité de notre code, après chaque étape, on interagissait avec notre bot pour déceler les erreurs et les corriger immédiatement avant de passer à l'étape suivante. A part, une traduction erronée (ci-dessous), nous n'avons décelé aucun problème pour le moment.



Conclusion et Perspectives

1. **Résultats :**

   Description des fonctionnalités finales implémentées

   Notre bot ne possède plusieurs fonctionnalités.

   - **Initialisation et Démarrage**
     Le bot peut être démarré avec la commande « `/start` », qui accueille l'utilisateur et explique comment utiliser le bot.
   - Ensuite, l'utilisateur défini sa langue. Puis celle dans laquelle il souhaite traduire son message.
   - **Réception et Transcription de Messages Vocaux**
     Le bot peut recevoir des messages vocaux, télécharger le fichier audio, le convertir en format WAV et transcrire le contenu vocal en texte.
   - **Traduction du texte**
     Après la transcription, le bot traduit le texte transcrit de la langue source à la langue cible spécifiée par l'utilisateur.
   - **Nettoyage des Fichiers Temporaires**
     Après le traitement du message vocal, le bot supprime les fichiers audios temporaires pour éviter d'encombrer le système.

Apport Pratique pour les Utilisateurs de la Plateforme

Notre Bot offre plusieurs avantages pratiques pour les utilisateurs de la plateforme Telegram :

Facilité de Communication Multilingue :

Grâce à la fonctionnalité de transcription et de traduction intégrée, le bot permet aux utilisateurs d'envoyer des messages vocaux dans leur langue maternelle et de recevoir des traductions dans la langue souhaitée, simplifiant ainsi les échanges.

Accessibilité et Gain de Temps :

Le bot permet aux utilisateurs de simplement enregistrer un message vocal, qui est ensuite automatiquement transcrit et traduit, ce qui économise du temps et des efforts.

Support Multilingue Amélioré :

Le bot peut être utilisé pour transcrire et traduire les messages vocaux des clients, facilitant ainsi un support plus efficace et réactif dans diverses langues.

Intégration et Convivialité :

Le bot offre une solution intégrée et conviviale directement dans Telegram, une plateforme déjà largement utilisée, sans nécessiter de configuration supplémentaire pour les utilisateurs.

2. **Leçons Apprises** :

Durant la création de notre, nous avons eu à faire face à plusieurs problèmes dont :

- L'obtention des API OPENIA gratuites. Nous avions voulu utiliser les bibliothèques d'openia mais malheureusement nous n'avons pas pu obtenir une api. Donc nous avons utilisé les bibliothèques speech_recognition et translate à la place.
- Au départ, notre bot ne prenait en compte que l'anglais pour la transcription et ne traduisait qu'en français. Pour mettre à l'utilisateur de choisir les langues, nous avons crée un dictionnaire de langue et nous avons mis en place des boutons qui suggèrent les langues disponibles. Ainsi l'utilisateur n'aurait qu'à toucher les langues de son choix.
- Nous n'arrivons pas à faire exécuter correctement notre script avec le logiciel « Visual Studio Code », nous sommes constamment confrontés à des erreurs dont nous n'arrivons pas à résoudre. Finalement, nous avons préféré utiliser « Jupyter ».

Au cours de ce projet nous avons acquis plusieurs compétences dont : l'obtention d'une API et la création d'un bot telegram, l'utilisation des bibliothèques de notre script, et aussi nous avons développé nos compétences en programmation python et en gestion des erreurs.

3. **Perspectives d'Amélioration** :

Nous travaillons actuellement sur une fonctionnalité permettant de convertir la transcription de messages vocaux en audio traduit. De plus, nous développons une option offrant à l'utilisateur le choix du format de retour : uniquement le texte transcrit, le texte traduit, l'audio traduit, ou une combinaison de ces éléments.

Nous prévoyons notamment l'intégration de notre bot :

- Avec des Outils de Productivité comme **Google Drive / Dropbox :** Permettant ainsi aux utilisateurs de sauvegarder automatiquement les transcriptions et traductions dans leur compte Google Drive ou Dropbox pour un accès facile et sécurisé.
- Avec des Applications de Messagerie comme **Slack / Microsoft Teams :** pour faciliter la communication multilingue au sein des équipes.
- Avec l'utilisation de Services de Traduction Avancés comme **API de Google Translate ou Microsoft Translator :** Intégrer des services de traduction plus robustes pour améliorer la précision et la diversité des langues prises en charge par notre bot.
- Avec des outils d'analyses et Statistiques comme **Google Analytics / Tableau :** pour surveiller l'utilisation du bot et obtenir des insights sur les interactions des utilisateurs.

4. **Annexes** :

- https://core.telegram.org/bots/api?form=MG0AV3 : Documentation officielle de l'API Telegram pour les bots.
- https://core.telegram.org/bots/tutorial?form=MG0AV3 : Tutoriel officiel de Telegram pour créer un bot avec Python.
- https://realpython.com/python-speech-recognition/?form=MG0AV3 : Un tutoriel détaillé sur l'utilisation de la bibliothèque speech_recognition en Python.
- https://docs.python-telegram-bot.org/en/v21.10/?form=MG0AV3 : Bibliothèque utilisée pour interagir avec l'API Telegram et gérer les commandes et messages.
- https://docs.python-telegram-bot.org/en/stable/telegram.ext.html?form=MG0AV3 : Sous-bibliothèque de telegram fournissant des extensions et des outils supplémentaires pour faciliter la gestion des commandes, des messages, et des filtres.
- https://pydub.com/?form=MG0AV3 : Bibliothèque utilisée pour manipuler les fichiers audios, notamment pour convertir les fichiers OGG en WAV.
- https://pypi.org/project/translate/?form=MG0AV3 : Bibliothèque utilisée pour traduire le texte d'une langue à une autre.
- https://docs.python.org/3/library/os.html : Bibliothèque standard de Python utilisée pour interagir avec le système d'exploitation, notamment pour la gestion des fichiers et des variables d'environnement.
- https://docs.python.org/3/library/logging.html : Bibliothèque standard de Python utilisée pour générer des logs et enregistrer les événements et erreurs.
- https://pypi.org/project/nest-asyncio/ : Bibliothèque utilisée pour patcher l'asyncio event loop, permettant son utilisation dans des environnements comme Jupyter Notebook.
- https://pypi.org/project/python-dotenv/ : Bibliothèque utilisée pour charger les variables d'environnement depuis un fichier .env, permettant de sécuriser les informations sensibles.